



A Criptografia RSA e seus Fundamentos Matemáticos

Rafael Luiz Gonçalves dos Santos

Universidade Federal de Uberlândia
Faculdade de Computação - FCOM
rafael.lui@ufu.br

Mateus Teixeira Silva

Universidade Federal de Uberlândia
Faculdade de Computação - FCOM
mateusteixeira1512@ufu.br

Nicolly Ribeiro Luz

Universidade Federal de Uberlândia
Faculdade de Computação - FCOM
nicolly.luz@ufu.br

Eduardo Rogerio Favaro

Universidade Federal de Uberlândia
Instituto de Matemática e Estatística - IME
eduardofavaro@ufu.br

Introdução

O método de criptografia de chave pública RSA foi introduzido por Rivest, Shamir e Adleman em 1978 [2] e tornou-se um dos pilares da segurança moderna, sendo um dos sistemas criptográficos mais utilizados em aplicações comerciais, com aplicações que vão de protocolos de transporte seguro a assinaturas digitais em uma ampla variedade de sistemas. Entre as aplicações, destaca-se o HTTPS/TLS (com amplo histórico de uso de RSA para troca de chaves até o TLS 1.2 e, ainda hoje, para autenticação por certificados), S/MIME para e-mail seguro, e autenticação em SSH, entre outros cenários de assinatura digital e sigilo de dados [3]. A segurança deste método criptográfico baseia-se na dificuldade de fatorar um número inteiro muito grande e em fundamentos da teoria de números, como aritmética modular, primalidade e o algoritmo euclidiano/estendido, que sustentam a geração de chaves, a cifra por exponenciação modular e a recuperação da mensagem pelo inverso modular. A dificuldade em fatorar inteiros com muitos dígitos justifica o uso de módulos com comprimento suficiente para resistir a ataques.

Para implementação e interoperabilidade do RSA, o PKCS #1 v2.2 (RFC 8017) [3] normatiza tipos de chaves, conversões entre octetos e inteiros (OS2IP/I2OSP), primitivas de cifra/decifra e assinatura/verificação, além de esquemas modernos como RSAES-OAEP (cifração) e RSASSA-PSS (assinatura), recomendados para novas aplicações.

Como este trabalho tem um enfoque mais didático, focando nas etapas matemáticas e em uma implementação ilustrativa em Python; quando pertinente, são apontados paralelos com recomendações do PKCS #1 v2.2, preservando a distinção entre uma exposição conceitual e requisitos normativos do RSA.

1. Pré-codificação

1.1 Transformando mensagem em uma sequência de números

O passo inicial do método RSA é a pré-codificação, que converte a mensagem de texto em uma sequência numérica. A implementação descrita utiliza uma tabela de conversão específica do livro "Números Inteiros e Criptografia RSA" [1], em vez de alternativas como a tabela ASCII. O processo é demonstrado com a frase "Paraty é linda", ignorando a acentuação.

1.2 Gerando números primos

A segunda etapa na preparação do sistema RSA consiste na geração de dois números primos extremamente grandes, com 150 algarismos ou mais. O livro explora diversas técnicas para encontrar primos, incluindo fórmulas como as dos

números de Fermat e de Mersenne. No entanto, para a finalidade de gerar as chaves do RSA, conclui-se que esses métodos baseados em fórmulas não são os mais eficientes ou práticos. A abordagem recomendada pelo livro é, na verdade, gerar números aleatórios gigantescos e ímpares e, em seguida, testar se eles são primos. Para realizar esta verificação de primalidade, o texto sugere a utilização de um teste probabilístico, citando o Teste de Miller-Rabin como o método a ser utilizado.

1.3 Teste de Primalidade de Miller-Rabin

O Teste de Primalidade de Miller-Rabin é um método eficiente para verificar a primalidade de grandes números, ideal para o sistema RSA, pois não depende da fatoração. O teste funciona realizando “testes rápidos” baseados em propriedades da aritmética modular. Para um número ímpar n , ele primeiro decompõe $n - 1$ na forma $n - 1 = 2^k \cdot q$, onde q é ímpar e, em seguida, analisa uma sequência de potências modulares.

É crucial entender sua natureza probabilística: o teste não garante com certeza absoluta a **primalidade** de um número. Contudo, uma falha no teste prova inequivocamente que ele é **composto**. Ao passar no teste por k bases aleatórias, a probabilidade de o número ser composto é no máximo 4^{-k} . Assim, tomando várias bases aleatórias, a probabilidade torna-se tão baixa que ele pode ser considerado primo para fins criptográficos com total segurança. ¹

```
1 def teste_miller_rabin(self, n, k_testes=5):
2     if n == 2 or n == 3:
3         return True
4     if n <= 1 or n % 2 == 0:
5         return False
6     # Executa o teste várias vezes com diferentes testemunhas
7     for _ in range(k_testes):
8         if not self._miller_rabin_passo_a_passo(n):
9             return False # Se falhar uma vez, é definitivamente composto
10    return True # Se passar em todos os testes, é muito provavelmente primo
11 def _miller_rabin_passo_a_passo(self, n):
12    # 1. Decompor n-1 em 2^k * q
13    q = n - 1
14    k = 0
15    while q % 2 == 0:
16        q //= 2
17        k += 1
18    # 2. Escolher uma testemunha aleatória 'a'
19    testemunha = random.randint(2, n - 2)
20    # 3. Calcular x = testemunha^q mod n
21    x = pow(testemunha, q, n)
22    # 4. Verificar a primeira condição: se x for 1 ou n-1, o número passa no teste
23    if x == 1 or x == n - 1:
24        return True
25    # 5. Se não, continuar elevando ao quadrado k-1 vezes
26    for _ in range(k - 1):
27        x = pow(x, 2, n)
28        if x == n - 1:
29            return True
30        if x == 1:
31            return False
```

Listagem 1: Implementação do Teste de Miller-Rabin em Python

1.4 O Processo de Blocagem da Mensagem

O processo de **blocagem** divide a longa mensagem numérica em blocos menores. A regra fundamental é que cada bloco deve ser numericamente **menor que a chave pública n** .

A estratégia de implementação descrita agrupa sequencialmente os códigos de 2 dígitos da mensagem, formando o maior bloco possível que ainda respeite essa condição. Quando um bloco não pode mais crescer sem exceder o valor de n , ele é finalizado e um novo é iniciado. Observa-se que a blocagem em pares de dígitos foi utilizada para fins didáticos; por [3], a blocagem deve ser em octetos para não introduzir inconsistências numéricas e problemas de reagrupamento na decodificação. ²

```
1 def pre_blocagem(self, frase):
```

```

2     chave_publica = self.gerar_chave_publica()
3     frase_pre_codificacao = self.pre_transforma_frase(frase)
4     # A string agora é uma lista de códigos de 2 dígitos
5     lista_de_codigos = [frase_pre_codificacao[i:i+2] for i in range(0, len(frase_pre_codificacao),
6                             2)]
7
8     blocos = []
9     bloco_atual = ""
10    for codigo in lista_de_codigos:
11        # Tenta adicionar o próximo código de 2 dígitos ao bloco atual
12        bloco_potencial = bloco_atual + codigo
13        if int(bloco_potencial) < chave_publica:
14            bloco_atual = bloco_potencial
15        else:
16            # O bloco potencial ficou muito grande, então salvamos o anterior
17            blocos.append(bloco_atual)
18            # O código atual inicia um novo bloco
19            bloco_atual = codigo
20    # Adiciona o último bloco que estava sendo formado
21    if bloco_atual:
22        blocos.append(bloco_atual)
23    print(blocos)
24    return blocos

```

Listagem 2: Implementação do processo de blocagem da mensagem

1.4.1 A Justificativa para a Blocagem

A blocagem é uma etapa matematicamente fundamental, pois o sistema RSA opera com base na **aritmética modular** de módulo n .

As fórmulas de codificação ($C \equiv M^e \pmod{n}$) e decodificação ($M \equiv C^d \pmod{n}$) exigem que a mensagem M a ser processada seja um número **menor que n** .

Como uma mensagem completa geralmente ultrapassa esse valor, a blocagem a divide em segmentos menores. Isso garante que cada bloco possa ser criptografado individualmente de forma correta e reversível dentro das regras da aritmética modular.

2. Codificação

Agora que as etapas de pré-codificação foram abordadas, vamos tratar da fase de codificação. Antes de chegar à implementação da codificação RSA propriamente dita, é preciso compreender um algoritmo que o livro apresenta como fundamental para este processo: o **Algoritmo Euclidiano**.

Este algoritmo é de extrema importância, pois seu objetivo é calcular o **máximo divisor comum (MDC)** entre dois números inteiros.

2.1 Funcionamento do Algoritmo

O Algoritmo Euclidiano calcula o Máximo Divisor Comum (MDC) entre dois inteiros através de uma série de divisões sucessivas. O processo consiste em dividir o maior número pelo menor e, em seguida, dividir o divisor anterior pelo resto obtido. Esta operação é repetida até que a divisão resulte em um resto zero. O MDC é o último resto não nulo encontrado na sequência. Por exemplo, no cálculo do MDC(1234, 54), a sequência de restos é 46, 8, 6, e 2. Como a divisão seguinte resulta em resto 0, o processo para, e o MDC é 2.

2.2 A Função Totiente de Euler ($\phi(n)$) e a escolha do Expoente Público (e)

Antes de codificar, é necessário entender a **Função Totiente de Euler**, denotada por $\phi(n)$. Conforme detalhado no Capítulo 8 do livro de referência, esta função representa a quantidade de números inteiros positivos menores que n que são primos entre si com n . Para a criptografia RSA, onde $n = pq$ é o produto de dois primos distintos, a função é calculada por:

$$\phi(n) = (p - 1)(q - 1). \quad (1)$$

Após o cálculo de n e de $\phi(n)$, o próximo passo é definir o expoente público e . A condição fundamental é que e seja coprimo com $\phi(n)$, ou seja, o Máximo Divisor Comum (MDC) entre eles deve ser 1, isto é, $\text{mdc}(e, \phi(n)) = 1$.

Um valor adequado para e é escolhido por meio de um algoritmo iterativo: testa-se inteiros, geralmente a partir de 2, calculando o MDC de cada um com $\phi(n)$ através do Algoritmo Euclidiano. O primeiro inteiro que satisfizer a condição de MDC igual a 1 é escolhido como o expoente e . A chave pública é o par (n, e) .

2.3 A Operação de Codificação

Uma vez que a chave pública completa, o par (n, e) , foi definida, a codificação pode ser realizada. Cada bloco M da mensagem pré-codificada é transformado em um bloco codificado C através da operação de **potenciação modular**:

$$C \equiv M^e \pmod{n}$$

Isso significa que cada bloco da mensagem é elevado à potência e , e o resultado final da codificação é o **resto da divisão** desse valor pela chave pública n .

```
1 def codificar(self, frase):
2     mensagem_blocada = self.pre_blocagem(frase)
3     bloco_codificado = []
4     e = 1
5     mdc = 0
6     o_n = (self.__keyPrivate1 - 1) * (self.__keyPrivate2 - 1)
7     while mdc != 1:
8         e += 1
9         mdc = self.algoritmo_euclidiano(e, o_n)
10    self.__e = e
11    for bloco in mensagem_blocada:
12        bloco_codificado.append(str(pow(int(bloco), e, self.__keyPublic)))
13    self.__fraseCodificada = ' '.join(bloco_codificado)
14    return self.__fraseCodificada
```

Listagem 3: Implementação da lógica da codificação

3. Decodificação

Agora que já falamos sobre todas as etapas e funções da codificação, vamos abordar o processo de decodificação. Antes de detalharmos a decodificação em si, é fundamental introduzir um algoritmo de extrema importância para sua implementação: o **Algoritmo Euclidiano Estendido**. A seguir, explicaremos seu funcionamento e como ele se aplica diretamente na decodificação.

3.1 O Algoritmo Euclidiano Estendido e a Chave de Decodificação (d)

O Algoritmo Euclidiano Estendido é uma poderosa variação do algoritmo euclidiano padrão. Dados dois inteiros a e b , ele apresenta dois inteiros, α e β , tais que

$$\alpha \cdot a + \beta \cdot b = \text{MDC}(a, b).$$

Com isto, podemos encontrar o inverso modular d de e , módulo $\phi(n)$, isto é,

$$e \cdot d \equiv 1 \pmod{\phi(n)}.$$

A **chave privada**, utilizada para decodificação é o par (n, d) . Note que, enquanto n é a mesma chave pública usada na codificação, d é uma **chave privada e secreta**. A seguir, vamos detalhar o processo de decodificação.

3.2 A Operação de Decodificação

Uma vez que o destinatário possui a chave secreta d , a decodificação de cada bloco criptografado C para o seu bloco original M é feita através da **potenciação modular**, de forma análoga à codificação:

$$M \equiv C^d \pmod{n}$$

Isso significa que cada bloco da mensagem codificada é elevado à potência d , e o resultado é o **resto da divisão** desse valor pela chave n , recuperando assim o bloco numérico original **4**.

```
1 def decodificar(self):
2     mensagem_codificada = self.__fraseCodificada
3     mensagem_blocada = mensagem_codificada.split(' ')
4     bloco_decodificado = []
5     o_n = (self.__keyPrivate1 - 1) * (self.__keyPrivate2 - 1)
6     x = self.algoritmo_euclidiano_estendido(self.__e, o_n)
7     d = int(x) % o_n
8     # Decodifica cada bloco para seu valor numérico original
9     for bloco in mensagem_blocada:
10         if bloco:
11             valor_decodificado = pow(int(bloco), d, self.__keyPublic)
12             bloco_decodificado.append(str(valor_decodificado))
13     string_numerica_completa = "".join(bloco_decodificado)
14     dicionario_valores = {str(v): k for k, v in self._dicionario_letras.items()}
15     mensagem_decodificada_final = ""
16     i = 0
17     while i < len(string_numerica_completa):
18         codigo_letra = string_numerica_completa[i:i+2]
19         if codigo_letra in dicionario_valores:
20             mensagem_decodificada_final += dicionario_valores[codigo_letra]
21         i += 2
22     return mensagem_decodificada_final
```

Listagem 4: Implementação da lógica da decodificação

Conclusão

Este trabalho detalhou o funcionamento do sistema de criptografia RSA, desde a conversão de uma mensagem em texto para uma sequência numérica até os processos de codificação e decodificação. Foi demonstrado como cada etapa do algoritmo se baseia em conceitos fundamentais da teoria dos números, como a geração de primos com o Teste de Miller-Rabin, o cálculo do MDC pelo Algoritmo Euclidiano, e a obtenção do inverso modular através do Algoritmo Euclidiano Estendido. A conexão entre a teoria matemática e a aplicação prática evidencia a elegância e a robustez do RSA, cuja segurança reside na dificuldade computacional de fatorar números inteiros grandes. A implementação didática em Python serve como uma ponte, solidificando a compreensão de como esses pilares matemáticos se unem para criar um dos sistemas criptográficos mais influentes da atualidade.

Código da Implementação

O código-fonte completo da implementação em Python, que ilustra os conceitos discutidos neste trabalho, está disponível no seguinte repositório: https://github.com/Rafarockdf/IC_CriptografiaRSA/blob/main/RSA_IC.py

Agradecimentos

Agradecemos ao CNPq e a FAPEMIG pela oportunidade e suporte financeiro através das bolsas de iniciação científica dos alunos envolvidos neste projeto.

Referências

- [1] Coutinho, S. C. *Números Inteiros e Criptografia RSA*, Editora Impa, Rio de Janeiro, 2009.
- [2] Rivest, R.L., Shamir, A. e Adleman, L. (1978) A method for obtaining digital signatures and public-key cryptosystems, *Comm. ACM*, 21, 120-126. Disponível em <https://people.csail.mit.edu/rivest/Rsapaper.pdf>. Acesso em: 3 set. 2025.
- [3] MORIARTY, K. et al. PKCS #1: RSA Cryptography Specifications Version 2.2. Internet Engineering Task Force, nov. 2016. (Request for Comments: 8017). Disponível em: <https://datatracker.ietf.org/doc/html/rfc8017>. Acesso em: 3 set. 2025.